

Reading Course Reports

From *Swarm Robotics – A Formal Apporach* and *Review Papers*

By Yeqi Sang

Chapter 1 Summary: An Overview of Swarm Robotics

By Yeqi Sang

1.1 Introduction to Swarm Robotics

Swarm robotics is a field without an explicit definition in the literature. Instead of being strictly-defined, swarms are characterized by their behaviors. A system can be considered a swarm if it implements swarm behavior. Properties and requirements of swarm behaviors are discussed in the following.

1.2 Characteristics of Swarm Robotics

Swarm robotics emphasizes collaboration among robots to achieve performance enhancements beyond simple parallelization. Key properties of robotic swarms include:

1. **Decentralized control:** Each unit controls itself and the control paradigm prohibits the controls of a unit by other units.
2. **Asynchronicity:** The swarm lacks a global clock for swarm members to access. A swarm has to explicitly synchronize if necessary.
3. **Homogeneity:** Swarms typically consist of simple, quasi-identical robots that are often mass produced.

The properties contribute to robustness, fault-tolerance and fail-safety of swarm systems. Each robot can be substituted by any other. Due to redundancy and the avoidance of single-point-failure, swarms are inherently reliable, flexible and scalable.

1.3 Density, Performance and Communication in Swarm Robotics

The whole size usually cannot represent the swarms. Density plays a critical role in swarm performance, considering the performance, communication. Normally it will have four stages, with the increase of density.

1. **Sub-linear Increase:** At low density, communication and collaboration are limited.
2. **Super-linear Increase:** Performance improves as density increases, enabling more efficient collaboration.
3. **Optimal Swarm Density:** Collaboration reaches peak efficiency with max performance.
4. **Performance Decline:** Excessive density leads to interference, communication overheads and a rapid drop in performance

Communication within swarms can be:

1. **Explicit:** Direct exchange of information between robots.
2. **Implicit:** Indirect communication through environment cues. E.g.: One robot can detect the trail of another robot ahead to make a decision.

1.4 Micro and Macro: Self-Organization and Emergence

Swarm robotics operates at two levels, Micro and Macro. The existence of these two levels implies a grand challenge for the design of swarm robotics systems. The task/behavior of the swarm is defined on the macro level. While the implementation of the robot controller is to be done on micro-level, for example, executable code for the robot's microcontroller.

The idea of self-organization explains exactly this relation between micro and macro and gives an answer to the question how macro-structures can be generated based on micro-interactions only. Self-Organization can be described as a generator of “order from noise”, based on four components: positive feedback, negative feedback, multiple interactions and balance of exploitation and exploration, where the most important two of four are feedbacks.

1. **Positive feedback** is a force that increases, inflates and escalates quantities. It helps to break the initial randomly uniform distribution of system properties and reinforce small deviations.
2. **Negative feedback** plays a role as a protection mechanism, to limit the system so that the system can operate on finite resources. It is a force that reduces, deviations, diminishes majorities and damps overshoots.

Emergence is a key property in complex systems, showing that the behavior of the complex system cannot be understood by examining only the components of the system. Due to the interactions of the system components, the resultant system may be unpredictable.

The local behavior of an individual robot is defined by the programmer via the robot’s control program without explicitly defining the emergent, global behavior which cannot be perceived by an individual robot. Emergence is an intriguing concept because it constitutes the construction of a complex system by an unproportionally small effort.

1.5 Comments

Density is the most important property I learned from Chapter 1. With the introduction of density, the author highlights related properties such as robustness, scalability, and flexibility.

Next, the chapter discusses the micro-level and macro-level concepts. It further introduces self-organization, explaining how swarms can adapt in a closed-loop system using feedback mechanisms. Emergence demonstrates that swarm behaviors cannot be precisely predicted by examining individual robots alone. Due to the interactions within the swarm system, the overall system becomes significantly more complex and sometimes produces unexpected or surprising outcomes.

The interface between human engineers and robot swarms is highly beneficial and, in some cases, essential. It is crucial to enable simple debugging of swarms without requiring individual robots to be handled manually.

Many tasks have been completed by robot swarms using their motion and ability to detect other swarm members. The list of potential tasks is extensive and not necessarily different from those that can be accomplished by a single robot. However, a swarm of robots offers significant advantages by parallelizing task execution and enhancing qualities such as robustness and scalability. In future applications, swarms can be utilized in hazardous environments or for tasks requiring autonomous systems and high levels of parallelization, such as cleaning plastic waste from oceans or removing space debris threatening satellites.

Chapter 2 Summary: Short Introduction to Robotics

By Yeqi Sang

2.1 Introduction

This chapter provides a short introduction to mobile robotics starting from fundamental concepts such as sensors, actuators and kinematic. Then the author continued with a short introduction to open-loop and closed-loop control. Agent models, behavior-based robotics and potential field controls are included as control options, particularly interesting for swarm robots. Finally, the author showed several hardware platforms dedicated to swarm robotics, describing the challenge these platforms might have.

2.2 Fundamental Concept of Robots

Generally, we distinguish three classes of robots: mere manipulator, mobile robots and hybrid robots. Mobile robots are the main focus. They are unmanned vehicles that are capable of locomotion, with proximity sensors.

A robot has effectors, actuators, sensors, a controller and software. In a complex robot, effectors can be a hand to grip objects, but more common would be a torch, wheels, or legs. Actuators can be electric motors, piezoelectric motor or any other device that allows the robot to apply a physical force on command. Also, the body of a robot can be understood as a graph of links and joints. To configure the robot correctly, we need to know the degree of freedom and the constraint from the robot.

2.3 Sensors

Sensors come in two categories: active and passive sensors. Infrared and sonar sensors are active sensors, they also belong to the class of rangefinders. The problem with active sensors used as rangefinders is interference. Sometimes a sensor cannot distinguish between echos of its own signal and signals from other sensors. Interference is a challenging problem and there is no simple solution. However, one option is to limit the range of used sensors, hence scaling the problem down.

Imaging sensors are an example for passive sensors. They create a visual representation of the robot's environment. With stereo vision systems, the robot can also get depth information. Another class of passive sensors are proprioceptive sensors. These are sensors that gather information about the robot's internal states, for example, positions of joints. A typical example is a shaft decoder that counts the revolutions of an axis (wheel axis normally).

2.4 Odometry

Odometry is the estimation of distance and direction from a previous position of a robot. While it also introduces dead reckoning into the system, that is the art of determining the position based on a known initial position and subsequent headings and speeds. Uncertainty is an issue in odometry. Errors will sum up and the determined positions may be different from the robot's actual position, even if the robot has perfect information about its initial position and measure the revolutions of its axes with proprioceptive sensors. (GPS is not always a simple answer due to the requirement of higher accuracy or indoor operations).

2.5 Errors and Control

In odometry there are two classes of errors depending on their source. There are non-systematic errors that can usually not be measured and hence cannot be included in a model. These errors can be caused by uneven friction of the surface, wheel slippage, etc. Systematic errors are caused by differences between the model that the robot has of itself and the actual behavior of the robot.

In classical robotics there are two main controller paradigms: open-loop control and closed-loop control. An open-loop controller executes the robot's movement without paying attention to any feedback, that is, there is no chance to make corrections. The closed-loop controller executes the robot's movement and checks the progress with sensors. Hence, with closed-loop control, we can compensate for errors.

2.6 Controllers for Swarm Robots

The controllers here are more specific to be software control. The standard control approach in swarm robotics is hence based on reactive control. The simplest model is that of the simple reflex agent, based on simple condition-action rules (if-then rules) that determine how it behaves. The second agent model is the model-based reflex agent. This agent has an internal state that allows the robot to potentially react differently to the very same perception. The behavior of the robot is still determined by condition-action rules while considering a history by perceptions.

Potential field control is also an example of reactive control. The potential field is constructed based on the robot's perception and based on the defined task. Obstacles are represented by cones and the robot will move along the largest gradient of potential field. This is a reactive behavior because the robot does not plan its action and the robot's actions are directly determined by the currently present slope of the potential field.

2.8 Swarm Robot

Most examples of swarm robots are small mobile robots with differential drive and infrared sensors for proximity sensing and communication. Other special platforms relevant to swarm robotics are some approaches from modular robotics, where autonomous robot modules can physically attach to each other but also operate as independent entities.

For autonomous underwater vehicles, communication between robots is challenging underwater. Radio cannot be used because electromagnetic waves do not propagate far in water. Visible light and acoustic modem can be some of the solutions.

Swarm robotics from drones is also challenging. A reason may be that it is very hard to control multiple quadcopters within a small space, creating a lot of air turbulence, while having all of them rely on their onboard sensors exclusively.

2.9 Comments

In the book, it is unexpectedly noted that making robots move in a straight line is a challenging task. PID control is one of the closed-loop control approaches that allows the robot to make corrective adjustments. The difficulty arises not only from the complexity of swarm behavior but also from the challenge of identifying an effective controller. While the controllers themselves are relatively simple, determining the right one remains a difficult problem.

Chapter 3 Summary - A Simple Recipe of Control Method for Swarm Robotics

Yeqi Sang

1 Introduction

Chapter 3 is a quick walk through the methodology that is of interest to design swarm robot systems. It brings up a simple example, with the assumption that each robot has 2 neighbours. The author models a robot controller with a finite state machine for a collective-decision-making problem. It also extends the example to discuss whether the system represents a self-organizing system.

2 Minimal Example: Collective Decision-Making

In particular, we focus on situations when the states of neighboring robots determine each robot's state transitions and hence its behavior. Chapter 3 provides an easy-to-understand example to present a brief recipe. The following are the assumptions:

- There are only two possible states: A and B.
- The swarm size is N .
- Variable a is defined to count robots currently in state A. Similarly, variable b for state B.
- An agent is able to determine the internal state of all its neighbours and all robots keep moving around.
- $|n|$ is the number of neighbors.

In this example, both a and b reflect global knowledge about the swarm that is available to an external observer, but not to the robots themselves. Based on that, we can know:

- $N = a + b$ and we define $\alpha = \frac{a}{N}$, representing the fraction of robots that are in state A in a global view.

If We randomly pick a robot R_0 , then R_0 and its neighborhood consist of a sample of the swarm. Then we have

- Variables $\hat{a}$ and $\hat{b}$ as the number of neighbouring robots of robot R_0 plus itself that are in state A and state B. Thus, for R_0 and its neighbors, we can give a fraction of robots that are in A by:

$$\hat{\alpha} = \frac{\hat{a}}{|n| + 1} \quad (1)$$

This is also called "local sampling" when we focus on R_0 and its neighborhood. Generally we have $\hat{\alpha} \neq \alpha$. Our hope is that in average the local measurement $\hat{\alpha}$ gives a good approximation to the global state α in average and the system is unbiased. As a consequence, the whole swarm should converge on a consensus of either $\alpha = 1$ or $\alpha = 0$. Based on the above defined result $\hat{\alpha}$ of locally sampling the neighborhood and a threshold of 0.5, we can define a finite state machine for this little collective decision-making scenario.

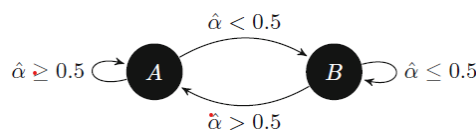


Figure 1: Finite State Machine for Collective Decision-Making

3 Macroscopic Perspective

For simplicity, we restrict ourselves to a small neighborhood of $|n| = 2$ and we make use of the above defined true swarm fraction α . To view how the finite state machine at micro level affect the macro level, we have the following relationship. The probability $P_{B \rightarrow A}$ to switch from B to A is given by

$$P_{B \rightarrow A}(\alpha) = (1 - \alpha)\alpha^2$$

This is because in total we have 3 robots. if the considered robot wants to transfer from B to A, that means its two neighbors are all in state A, with the assumption $\alpha = \hat{\alpha}$, $\alpha = \frac{2}{3} > 0.5$.

Similarly, the probability to switch from A to B is

$$P_{A \rightarrow B}(\alpha) = \alpha(1 - \alpha)^2$$

For the probability to switch from A to A, things get more complex. We need to consider that there are three cases from the robot transfer from A to A, that is AAA, ABA, AAB. The first A represents the considered robot, the rest of two represents its two neighbors. AAA means all three robots are already in state A. ABA and AAB means either of the neighbours are in state. Thus we have:

$$P_{A \rightarrow A}(\alpha) = \alpha^3 + 2(1 - \alpha)^2$$

Similarly, for the probability to switch from B to B, we have:

$$P_{B \rightarrow B}(\alpha) = (1 - \alpha)^3 + 2\alpha^2$$

4 Expected Macroscopic Dynamics and Feedbacks

We may want to know how does the swarm fraction α develop over time. For that we need to introduce a representation of time. We define interval Δt that is long enough to observe state transitions and short enough to observe not too many state traditions. Thus we get:

$$\frac{\Delta\alpha(\alpha)}{\Delta t} = \frac{1}{N} ((1 - \alpha)\alpha^2) - \frac{1}{N} (\alpha(1 - \alpha)^2). \quad (2)$$

The first term represents transitions $B \rightarrow A$, which contribute to generating more robots in state A. The second term represents transitions $A \rightarrow B$, then the symbol is negative, representing to generate more robots in state B. Factor $1/N$ accounts for an assumed switching rate of one robot per time step.

While, the current system is still not self-organizing system since it only contains positive feedback. If all robots in state A, then there is no chance for any robot to transit to state B, in the current system. Thus, the system is not self-organizing.

To include negative feedback, we can allow robots to spontaneously switch between states. In that case, the swarms in average is always dynamic with state changes and we can calculate two boundaries of α based on the values of negative feedback.

5 Comments

The state machine for collective decision-making operates at a micro level. However, to link it with a macroscopic perspective, Chapter 3 assumes that the system is unbiased. I believe this will play a significant role in later chapters, particularly in developing effective approximations of the global view based solely on local sampling.

Factor $\frac{1}{N}$ initially doesn't make sense to me until I realized that without the factor, $\frac{\Delta\alpha(\alpha)}{\Delta t}$ will change based on the size of the swarm, which is terrible.

Summary of Chapter 4: Scenarios of Swarm Robotics

Author: Yeqi Sang

Chapter 4 is an extensive guide through the literature on swarm robotics. It explores various scenarios in swarm robotics, and categorizes these scenarios based on complexity, starting from simple behaviors and advancing toward more intricate cooperative tasks.

Aggregation and Clustering

In aggregation the task for the robots is to group themselves together based on the environments and local interactions. Clustering techniques are inspired by biological models such as insect and bacterial behavior. However, most swarm algorithms struggle with balancing exploration and exploitation, frequently get caught in local optima. Note that a generic optimal solution to the exploration-exploitation trade-off does not exist because it depends on the respective scenario. The robots are usually not synchronized, certain behaviors may create deadlocks, and the robots can hardly know when the exploration phase in the swarm is over.

Dispersion

Dispersion is the behavior directly opposite to aggregation. The distance between robots should be maximized but the robots should stay within communication range. This is a useful behavior for applications like surveillance and large areas monitored.

Pattern Formation, Object Clustering, Sorting, and Self-Assembly

Pattern formation involves robots organizing themselves into predefined shapes or modifying their environment accordingly.

In object clustering, robots transport items to group them together or categorize them based on features like color or size. This section introduces the concept of anti-agents—robots that behave differently from the majority—which studies suggest can enhance swarm system efficiency when present in optimal numbers.

The Brazil nut effect is referenced as an example of self-organization in sorting, emphasizing that self-organized sorting in nuts requires external energy input. While robots, unlike nuts, are physically homogeneous, they are programmed with varying collision avoidance radii to simulate size differences and prevent collisions at different minimal distances.

Self-assembly, on the other hand, occurs when robots physically link together, forming more complex structures or super-robots to tackle tasks beyond the capabilities of individual units.

Collective Construction

A key challenge in the cellular-automaton approach, and in collective construction as a whole, is translating a global blueprint into simple local rules. Additionally, collective construction often involves fundamental swarm behaviors like collective decision-making and task partitioning. Even for a single robot, ensuring that building blocks are placed without creating deadlocks is complex. When multiple robots construct in parallel, the risk of deadlocks increases further as they may obstruct each other's paths, making coordination crucial for efficiency.

Flocking and Collective Motion

Flocking refers to the coordinated movement of groups, inspired by bird flocks. Reynolds' three simple rules help simulate this behavior: (1) **Alignment** – adjust direction to match nearby individuals, (2) **Cohesion** – stay close to neighbors, and (3) **Separation** – avoid collisions when too close to others. In

swarm robotics, these principles enable robots to move in a stable and efficient manner, which is essential for effective navigation and collective motion.

Foraging

Foraging is a search and retrieval behavior inspired by biological strategies, particularly the ant method, where robots emulate pheromones to optimize path selection. Another approach involves forming a transport chain to efficiently move objects to the base station. A common challenge in foraging is traffic congestion at the target area, which can lead to interference between robots. Additionally, foraging systems can support heterogeneous swarm robots, enabling diverse roles and improved efficiency in resource collection.

Task Partitioning and Division of Labor

Effectively partitioning and allocating tasks within a swarm is a critical and complex challenge in swarm robotics, requiring various algorithmic strategies. Task partitioning should be self-contained, ensuring that each task has similar requirements and workload distribution. Division of labor allows robots to specialize, preventing redundancy. In dynamic environments, online task allocation is crucial, relying on local cues to assess worker distribution and task progress. While auction-based methods can be effective, they require all-to-all communication, limiting scalability. Alternatively, game theory-based approaches enable decentralized coordination using only local information and communication. Machine learning also presents a promising solution for optimizing task allocation in swarm systems.

Mixed Societies and Bio-Hybrid Systems

Mixed societies and bio-hybrid systems in swarm robotics involve the interaction between robots and living organisms. These systems can be leveraged to guide insect swarms, offering innovative solutions for pest control. By fostering a symbiotic relationship between robots and biological entities, such approaches combine the strengths of both, creating more efficient and adaptive systems.

Swarm Robotics 2.0

Swarm robotics is advancing in several key areas, including error detection and security, human-robot interfaces, and field robotics applications. Error detection and security are crucial, as partially functional robots can introduce errors and unexpected behaviors, potentially compromising the entire swarm. Detecting faults and isolating malfunctioning robots is essential to maintaining system reliability.

Human-robot interfaces are another emerging focus, particularly for human supervisors managing swarm systems. This growing field explores remote and proximal interaction, visualization, control mechanisms, levels of automation, and organizing sub-swarms to improve usability and coordination.

In field robotics, swarm systems encounter similar challenges as traditional robotics, particularly in achieving reliability. Developing robust robots often requires extensive sensors and computational power, but complex sensors tend to be too large, heavy, and expensive for swarm applications. Despite technological progress, current robots still fall far short of the capabilities of even simple insects.

Chapter 5 Summary: Modeling Swarm Systems and Formal Design Methods

Author: Yeqi Sang

1. Introduction

Chapter 5 introduces modeling as a dimension reduction technique in swarm robotics. It begins by addressing the challenge of dealing with local information through local sampling. Several modeling techniques commonly applied in swarm robotics are presented, including rate equations, spatial models based on ordinary and partial differential equations, network models, and biology-inspired methods.

Modeling of systems typically serves three key objectives: abstraction, simplification, and formalization. Abstraction involves omitting certain details of the modeled system to focus on essential components. Simplification transforms a complex system into a more interpretable version. Formalization describes the modeled system using a formal framework, enabling the derivation of new insights through logic and mathematics. These three steps are particularly important in swarm robotics, as such systems consist of many entities and are inherently high-dimensional.

Rather than working directly with high-dimensional data and states from the swarm, the goal is to understand the system's general behavior across different setups. This allows for dimension reduction through abstraction and simplification. In practice, the motion and observation equations of each agent in the swarm define a reduced configuration space that captures key features of the modeled system. This reduction facilitates efficient analysis and design while preserving critical swarm behaviors.

2. Local Sampling

When designing and programming controllers for swarm robotics, a common challenge is the lack of global information. Individual robots have only local perception and can gather limited additional data by communicating with nearby neighbors. Even highly capable agents face similar constraints in certain situations. For example, a human may still get lost if they only know the names of two adjacent streets but lack a full city map.

In swarm robotics, the only feasible approach is local sampling, as global communication does not scale efficiently. Robots can only share information with their immediate neighbors, limiting their knowledge to locally available data. In many cases, a robot relies solely on its own observations without additional communication. For instance, stigmergy in ant colony optimization enables agents to navigate using only local pheromone concentrations, which encode directional information. Another example is the alignment rule in flocking behavior, where a robot adjusts its movement by aligning with its neighbors' directions. This local sampling strategy provides a useful approximation of the swarm's dominant direction of motion. Moreover, self-organization can lead to emergent behaviors, making local sampling an effective strategy in swarm modeling.

Since sampling in swarm robotics is inherently local, it follows a nonprobability and accidental sampling process. This means a robot might be part of a localized cluster that is not representative of the entire swarm, or its state may be correlated with its neighbors, limiting independence in observations. To address this, Buffon's Needle approach provides a simple yet effective probabilistic method for estimating properties of the swarm environment without requiring specialized hardware. Inspired by a mathematical problem of estimating π by dropping needles on a plane, this approach can be adapted to swarm robotics for stochastic measurement and spatial analysis, helping to infer global properties from local interactions.

3. Modeling Approach

In swarm robotics, scientists are still searching for a general and universally applicable modeling technique. While space can sometimes be abstracted away, in certain cases—such as pattern formation—spatial information remains essential. Below is an overview of key modeling approaches, though the list is not exhaustive.

Rate equation models are commonly used in chemistry to describe reactions between reactants. In swarm robotics, this concept is adapted by replacing chemical concentrations with fractions of robots in specific states. A robot's state transitions, defined within its state machine, may be triggered by encountering another robot in a certain state, akin to reactants interacting in a reaction. This approach is macroscopic, probabilistic, and non-spatial, relying on a well-mixed assumption where all interactions occur with uniform probability.

Unlike rate equations, differential equation models explicitly incorporate space, making them suitable for modeling the movement and distribution of swarm robots. One key challenge in spatial modeling is distinguishing between active and passive movement as well as purposeful vs. random behavior. Chapter 5 introduces the Fokker-Planck equation, which provides a macroscopic representation of a swarm's probabilistic density distribution over time. This equation is closely related to the Langevin equation, which describes microscopic individual movement, incorporating noise and external forces. The significance of these equations lies in their mathematical connection: the Fokker-Planck equation describes how probability densities evolve at the macroscopic level, while the Langevin equation governs individual trajectories at the microscopic level. This relationship is crucial in swarm robotics because it allows researchers to bridge the gap between individual robot behaviors and the emergent collective dynamics of the swarm. By using the microscopic Langevin approach, one can derive the macroscopic Fokker-Planck equation, providing a theoretical foundation for understanding how local interactions scale up to system-wide patterns.

Swarm robotic systems can also be represented as networks, where robots act as nodes and connections between them indicate mutual neighborhood relations. A common modeling approach is to use random graphs with a disk-range connectivity model, which helps mitigate the issue of missing spatial correlations between nodes. However, these graphs are typically static, meaning they only represent snapshots of swarm configurations rather than capturing their dynamic evolution over time.

4. Formal Design Methods

The design of swarm robotics systems is challenging, even though individual behaviors are simple. In general, there are two main design approaches. A top-down approach begins by specifying the desired global system behavior, often assuming global knowledge, similar to a centralized system. In swarm robotics, this is decentralized by replacing global knowledge with local communication. In contrast, a bottom-up approach starts by defining the capabilities and constraints of individual agents, allowing global behavior to emerge through local interactions among agents and between agents and their environment.

To better manage the complexity of swarm design, multi-scaling models help bridge the gap between microscopic agent-level behaviors and macroscopic system-wide dynamics, enabling a structured transition between different levels of abstraction. Additionally, software engineering techniques such as modular programming and behavior-based architectures facilitate the development, testing, and maintenance of swarm control algorithms. Verification methods, including formal model checking and statistical validation, ensure reliability by systematically testing whether the swarm system meets predefined specifications. Lastly, global-to-local programming aims to translate high-level swarm objectives into local agent rules, ensuring that decentralized behaviors collectively achieve the intended global outcome.

Chapter 6 Summary: Collective Decision-Making

Author: Yeqi Sang

1. Introduction

Collective decision-making is a fundamental capability for enabling autonomy not only at the level of individual robots, but also for the swarm as a whole. Chapter 6 begins by reviewing traditional frameworks used to model decision-making among rational agents, then expands to explore how decisions can emerge from group interactions. Key models discussed include the urn model, voter model, and majority rule mechanisms. This summary will focus on these core models and conclude with a brief discussion on how collective decision-making can be implemented in real-world swarm robotics systems.

One of the central goals in swarm robotics is to build fully autonomous swarms. While individual autonomy is important, swarm-level autonomy is critical for emergent intelligent behavior. In such systems, there is no centralized control or global communication. Robots interact only with nearby peers, which introduces unique challenges. For instance, robots must recognize when a collective decision-making process is underway, avoid getting stuck in deadlock situations—sometimes without even realizing it—and determine when a consensus has been reached.

2. Decision-Making & Group Decision-Making

Decision-making can, in theory, be based on certainty. However, the environments in which robots typically operate are far more complex. As a result, decision-making in robotics is often carried out under conditions of uncertainty—which is, in fact, the core challenge of intelligent behavior. It is generally assumed that a robot must make decisions based on incomplete information about its environment, and that it can estimate the outcomes of its actions by probability. A rational decision is defined as one that optimizes expected utility given a set of constraints. In other words, the agent selects the action that yields the highest expected payoff, averaged over all possible outcomes.

When decisions are made by groups rather than individuals, collaborative or collective decision-making becomes necessary. This introduces additional complexity. For example, group choices can be subject to voting paradoxes, where the collective preference becomes inconsistent or contradictory despite individual preferences being rational. In animal swarms, collective decision-making is not always robust and, in some cases, may even lead to catastrophic failure.

In both natural and artificial swarms, collective decisions emerge from local interactions. Agents communicate only with their immediate neighbors or through indirect environmental cues. These systems are typically asynchronous: spatially distributed agents begin the decision process at different times and with different information. Local decisions based solely on local perception can lead to deadlocks, oscillations, or other unstable dynamics that complicate consensus-building.

3. Models for Collective Decision-Making Processes

A collective decision-making process typically consists of iterations over three main phases: exploration, dissemination, and opinion switching.

- In the exploration phase, a robot investigates an area associated with its current opinion and gathers information about its quality.
- During the dissemination phase, the robot communicates its opinion to nearby peers, either through explicit messages or indirect cues. For instance, the dissemination duration can be made proportional to the perceived quality — if the quality is doubled, the robot disseminates the opinion for twice as long. This coupling of dissemination time and opinion quality can trigger positive feedback loops, where better opinions receive more attention and are more likely to spread.
- In the opinion switch phase, each robot may change its opinion based on decision-making rules, such as the voter model or majority rule. Importantly, robots act asynchronously, each following its own internal clock.

Models such as the Urn Model, Ehrenfest Urn Model, Eigen Urn Model, and Swarm Urn Model illustrate how collective decision-making evolves:

- Initially, opinions may be distributed evenly (e.g., a 50:50 split).
- As interactions occur, the system tends to converge toward a dominant opinion.
- In more advanced models like the Swarm Urn Model, the feedback mechanisms vary depending on the system's state: strong positive feedback can stabilize a decision, while still allowing for flexibility and potential opinion reversal.

The Swarm Urn Model provides a simple macroscopic description of collective decision-making, while capturing microscopic aspects as well — such as how one robot can influence another. It also explicitly models the effects of positive and negative feedback, making it a powerful tool for analyzing and designing decision-making dynamics in robot swarms.

The Voter Model is a simple yet powerful framework for collective decision-making. In this model, an agent randomly selects one of its neighbors and adopts that neighbor's opinion. At first glance, this may seem trivial; however, the Voter Model has been shown to produce competitive results. Although the convergence process is relatively slow, it is also highly accurate, enabling the swarm to gradually reach a consensus that often reflects the correct or optimal decision.

In contrast, the Majority Rule Model operates by having each agent observe its local neighborhood and adopt the most frequent opinion among its neighbors. This model typically achieves consensus more quickly than the Voter Model. However, its accuracy tends to be lower, as the majority can form prematurely based on local biases or early random fluctuations.

4. Few Hardware Implementations & Comments

Several hardware implementations of collective decision-making have also been introduced. For example, experiments by Valentini et al. define collective perception as a form of decision-making, where the underlying logic closely mirrors that of traditional collective decision-making frameworks. Similarly, aggregation behavior can be interpreted as a decision-making process, as robots must choose among a set of possible aggregation sites. When these sites offer equal utility, or when multiple top options exist, the swarm faces a symmetry-breaking problem—a classic challenge in decentralized coordination.

The investigation, modeling, and analysis of collective decision-making is still a relatively young field, and continued exploration of diverse models is both necessary and promising. Topics such as opinion dynamics are gaining traction, and many studies have approached these problems using tools from statistical physics, further enriching the theoretical landscape.

In my view, collective decision-making remains highly task-specific. It is difficult to design a single, generic method that works across all scenarios—unless the system is capable of functioning with the adaptability and complexity of the human brain. After reading Chapter 6, I came to realize that each agent in a swarm functions much like a neuron in a brain. Just as neurons collectively process information and contribute to complex cognitive functions, perhaps robot swarms can one day perform sophisticated decision-making in a similarly distributed and intelligent manner.

Chapter 7 Summary: Case Study – Adaptive Aggregation

Yeqi Sang

1 Abstract

In this case study, the author integrates various techniques essential for designing a system of swarm robots. The task is relatively simple and focuses on **adaptive aggregation**—requiring the swarm to gather at a spot determined by environmental features. Following a biological role model, the author applies standard modeling techniques in swarm robotics. The control algorithm and the micro-macro model are ultimately validated against real robot experiments.

2 Experiment Setup

In this simplified scenario, the swarm is required to aggregate at a specific temperature value determined by scientists, allowing the system to adapt to a dynamic environment. The temperature distribution may contain local optima, i.e., multiple scattered peaks, rather than a single smooth maximum. However, for simplicity, the study assumes that the temperature distribution is always differentiable and has only a single global optimum with no local optima.

3 Alternative Solutions

3.1 Ad-hoc Approach

A simple method involves robots moving randomly while recording temperature measurements. Each robot could retain the highest recorded value and attempt to return to that location. However, this strategy has several drawbacks:

- It heavily relies on the precision of sensors.
- A faulty sensor reading can cause the robot to chase a non-existent maximum.
- The robot may end up in a deadlock, unable to locate the previously recorded maximum.

3.2 Gradient Ascent

An alternative method involves **gradient ascent**, where robots estimate the spatial derivative of the temperature field:

- The gradient must be well-defined and smooth.
- There should be no local optima.
- Reliable sensors are needed for detecting gradients.
- To calculate the gradient, robots must have either:
 - Two sensors to simultaneously measure at different positions, or
 - Memory to store past measurements as they move.

4 Modeling Aggregation

The author models both individual and swarm behaviors using a microscopic and macroscopic approach. The microscopic model captures individual robot behavior, while the macroscopic model is based on the **Langevin equation** and the **Fokker-Planck equation**. The model aims to:

- Capture the difference between ineffective individual behavior in flat gradient environments and effective swarm behavior.
- Represent agent algorithm parameters explicitly.
- Incorporate spatial features.
- Show quantitative agreement with empirical robot swarm experiments.

4.1 Microscopic Model

Each robot can be in one of two states: *moving* and *stopped*.

Here is the parameter explanation:

- $\mathbf{R}$: the robot position.
- $P(\mathbf{R})$: the associated temperature at position $\mathbf{R}$.
- $\nabla P(\mathbf{R}(t))$: the gradient of the temperature field at the robot's current position.
- F : represents the stochastic process that models the robot's random behavior.
- B : the control matrix defining how the random behavior affects the state.
- α : the hyperparameter to regulate the intensity of the robot's global-oriented behavior.
- $w(\mathbf{R})$: the waiting function for each robot.
- c : a scaling constant.

From general Langevin equation, we want to model the "flat gradient information" situation. We set $\alpha = 0$. This is a strong assumption because in order to achieve the desired aggregation at the warm spot, we then fully reply on an emergent, macroscopic effect. If the swarm can aggregate without individual goal-oriented behavior, then the robots do not need to detect difference in the temperatures anymore. Then the motion model is:

$$\dot{\mathbf{R}}(t) = \alpha \nabla P(\mathbf{R}(t)) + B\mathbf{F}(t) = B\mathbf{F}(t) \quad (1)$$

Following the standard approach of BEECLUST, the waiting function is:

$$w(\mathbf{R}) = \frac{w_{max} P^2(\mathbf{R})}{P^2(\mathbf{R}) + c} \quad (2)$$

When a robot encounters another robot, it transitions to the *stopped* state. The duration of stopping is governed by a waiting time function $w(R)$, which is dependent on the temperature value $P(R)$ at the robot's position R .

Each of the states is associated with an appropriate Langevin equation that models the robot's dynamics in that state. As described above, for state *moving* we have

$$\dot{\mathbf{R}}_m(t) = B\mathbf{F}(t) \quad (3)$$

For state *stopped* it is even simpler, as we have

$$\dot{\mathbf{R}}_s(t) = 0 \quad (4)$$

.

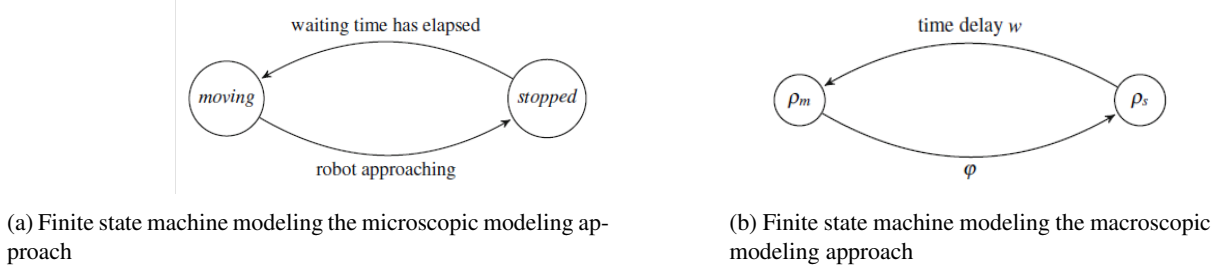


Figure 1: Micro-Macro Comparison

4.2 Macroscopic Model

For each microscope state we define a corresponding probability density. We define $\rho_m(\mathbf{r}, t)$ as the density of moving robots within an infinitesimally small area located at $\mathbf{r}$ at time t . Equivalently, we define $\rho_s(\mathbf{r}, t)$ as density of stopped robots at $\mathbf{r}$ at time t . For both densities we need a model of motion that is directly determined by the corresponding Langevin equation. For state *moving* we get

$$\dot{\mathbf{R}}_m(t) = B\mathbf{F}(t) \Rightarrow \frac{\partial \rho_m(\mathbf{r}, t)}{\partial t} = B^2 \nabla^2 \rho_m(\mathbf{r}, t) \quad (7.6)$$

That is a mere diffusion equation because we deleted any goal-oriented behavior already in the Langevin equation. For state *stopped* we get the expected simple result of

$$\dot{\mathbf{R}}_s(t) = 0 \Rightarrow \frac{\partial \rho_s(\mathbf{r}, t)}{\partial t} = 0 \quad (7.7)$$

The next step is to find a counterpart for the macroscopic model. The Fokker-Planck approach is continuous and the system needs to redefine the transitions. A master equation models the flow of probability density between different stages. The system need to decide the density flows from the *moving* density to the *stopped* density and back.

In case of the transition *moving* $\Rightarrow$ *stopped*, there is a relatively strong assumption saying that the probability of two robots approaching each other is directly proportional to the density of moving robots $\rho_m(\mathbf{r}, t)$. Hence, the system can define a stopping rate φ and the product $\rho_m(\mathbf{r}, t)\varphi$ gives the correct density flow of stopping robots. Then we can have the density of robots in the state *moving*:

$$\frac{\partial \rho_m(\mathbf{r}, t)}{\partial t} = B^2 \nabla^2 \rho_m(\mathbf{r}, t) - \rho_m(\mathbf{r}, t)\varphi + \rho_m(\mathbf{r}, t - w(\mathbf{r}))\varphi \quad (5)$$

where $\rho_m(\mathbf{r}, t)\varphi$ represent the robots in *stopped* state, and $\rho_m(\mathbf{r}, t - w(\mathbf{r}))\varphi$ represents the robots which stopped at timestamp $t - w(\mathbf{r})$ and now transit into the *moving* state.

Then for the density of robots in state *stopped*:

$$\frac{\partial \rho_s(\mathbf{r}, t)}{\partial t} = \rho_m(\mathbf{r}, t)\varphi - \rho_m(\mathbf{r}, t - w(\mathbf{r}))\varphi. \quad (6)$$

4.3 Initial Position

To numerically solve the PDE system that describes the robot density dynamics—particularly when time-delay terms such as $t - w(\mathbf{r})$ are involved—it is essential to define an appropriate initial state. This includes:

- A spatial distribution of robots in the *moving* state at time $t = 0$, i.e., $\rho_m(\mathbf{r}, 0)$.

- Due to the time-delay term, values of $\rho_m(\mathbf{r}, t)$ must also be specified for times **prior to** $t = 0$, i.e., for $t < 0$.

To simplify the initialization, a common assumption is:

$$\rho_m(\mathbf{r}, t) = 0 \quad \text{for } t < 0,$$

meaning that no robots were present in the system before the starting time.

Why is the initial state important?

- The partial differential equations (PDEs) used to model robot densities are time-dependent and include delay terms.
- These equations require knowledge of past system states to compute future evolution accurately—especially because of the term $\rho_m(\mathbf{r}, t - w(\mathbf{r}))$.
- Without an initial state (including pre-history), the system would be underdetermined, and numerical solvers could not proceed with time integration.
- Therefore, specifying initial conditions transforms the model into a well-posed initial value problem that can be solved numerically using methods such as Euler's method.

Summary - A review of swarm robotics tasks

Arthur: Yeqi Sang

1. Introduction

In this paper, various tasks for which past works have proposed solutions using a swarm intelligence approach are surveyed, focusing on distributed control, locality of interactions and simplicity of individual robot controllers. I will list tasks by different categories.

2. Aggregation and Flocking

For aggregation, the majority of swarm robotics proposing an algorithm to obtain aggregation in an artificial swarm used one of the following design method to control robot movement:

1. **Application of virtual forces:** either depends on distance or density between robots, may not be cost-effective with high error.
2. **Control of robot behavior based on a probabilistic approach:** unstable aggregates, small aggregates may prevent robots from joining larger aggregates.
3. **Artificial evolution:** neural networks linking sensory input to actuator outputs, need to escape from local maxima.

For aggregation algorithms, normally they are controlled by a finite state machine, with several states designed: approach, wait, repel.

Compared to aggregation, flocking has an important characteristic: alignment of robot movement, which allows a group to move collectively in a given direction. The usual assumption is that all robots have at least one neighbour which connects to the rest of the swarm. The algorithms generally have to deal with the conflicts between obstacle avoidance, flock formation and navigation, in decreasing priority order. In cases where only a fraction of robots have the knowledge of the global target, target direction is included as an additional component in the motion control input of the informed robots. Some strategies allow dealing with multiple goal directions with different priorities, so that when a higher-priority direction appears the robots informed of the new target are able to steer the swarm and override a lower-priority direction.

3. Foraging

In an artificial swarm robotics system for the foraging task, a specific area is designated as the “nest”, and the objective of the swarm is to find items scattered in the environment and bring them to the nest. Multi-foraging is an extension in which different types of items must be collected and each item is delivered to a nest specific place. (demining, hazardous waste cleanup, search and rescue and planetary exploration)

Although the target of foraging is collecting resources, the search activity entails an energy loss. In order to maximize the net energy income, the control algorithm of each robot is more relevant to dynamic task allocation between robots than to the foraging activity itself. Shared memory can help a swarm to form optimal paths in a more efficient way and reduce interference, either letting other robots follow, or repel other robots when the specified path is used too often.

4. Object clustering and sorting.

Compared to foraging, in the object clustering task, there is not a predefined destination place for collected objects, the goal being to place the objects near each other. Similar to foraging, in most object clustering scenarios, robots are assumed to be unable to sense the presence of distant objects, thus the typical behavior when searching for objects is to move randomly. Thus, the mechanism of communication between robots is usually not considered in clustering studies.

Most existing literature on clustering and sorting tasks makes use of simple robots with no localization capabilities, where clustering of objects emerges as a mere result of probabilistic interactions with the environment; in other studies, robots have the ability to localize themselves, and clustering is done with a more deterministic approach.

5. Navigation

A collective navigation scenario is one where a robot with limited sensing and localization capabilities is able to reach a target in an unknown location with the help of other robots. The key to a successful navigation strategy in a multi-robot scenario is sharing knowledge between robots. Such knowledge usually consists of an estimate of the distance to the target. Direct communication can significantly improve the efficiency of navigation since most robots do not have to play a role of landmark. Also, with shared information, various studies show the performance of a distributed navigation system is expected to be improved with later swarm size.

6. Deployment

In a self-deployment scenario, robots must deploy themselves in an environment without central coordinates. In a direct communication method, robots can either communicate explicitly, or implicitly by sensing the nearby presence and relative position of other robots. Stigmergic communication gives robots an indication of the action done by previous robots in the same area, normally using a positive feedback mechanism.

Most deployment can be divided into two main tasks: dispersion and pattern formation. In dispersion tasks, swarm members must position themselves away from the other, with the objective of maximizing the area covered globally by the swarm and/or minimizing the time elapsed. Additional constraints will be applied based on requirements, for example, the connectivity of the swarm must be preserved. Pattern formation generally has more constraints based on robots relative positions when viewed globally.

7. Collaborative Manipulation

Collaborative manipulation refers to swarm robotics tasks where groups of robots work together to manipulate objects in the environment. Two main types of object manipulation problems can be found in the existing literature: tasks where robots must collectively transport large objects, and tasks where sticks must be pulled from the ground.

In general, collaborative manipulation tasks require multiple robots to be located in the same place at the same time, and can be modeled as stochastic processes, whose dynamics is regulated by the characteristics of the environment and the robot control parameters. Probabilistic models for the stick pulling task are shown to provide good agreement with experimental results in characterizing the swarm performance.

8. Comments:

After reading the paper, I found that in swarm robotics, aggregation, flocking, and foraging serve as three fundamental operations, while other tasks such as object clustering, path formation, navigation, deployment, and collaborative manipulation are extensions or combinations of these core operations. These extended tasks either represent optimized solutions to specific problems or incorporate the fundamental operations as subcomponents of more complex behaviors.

One key takeaway from the paper is the role of obstacle avoidance mechanisms, where obstacles are modeled as virtual roots. Additionally, the limitation of sensors enhances scalability, making swarm systems more adaptable. Methods of flocking vary based on the level of global target knowledge, ranging from none to partially shared and fully shared knowledge. In foraging, efficient task allocation is as crucial as the foraging process itself, as maximizing net energy income is a primary goal. Shared resources among robots limit scalability, yet such systems can still enhance foraging performance.

Understanding performance metrics is essential for evaluating different swarm robotics tasks, particularly in clustering, where the inability of robots to determine their location and their reliance on perceiving objects within a limited range can prevent optimal placement of clustered objects. In this context, clustering dynamics emerge from positive feedback mechanisms, where actions performed at a given location increase the likelihood of repeated actions at the same site, reinforcing the clustering process.

Summary - A Review of Swarm Robotics in a NutShell

Author: Yeqi Sang

1. Introduction

This article provides a comprehensive overview of new research in the swarm robotics fields. A detailed review of the swarm's most emerging swarm behaviors, and swarm intelligence algorithms is captured, keeping in view the limitation and the transformation towards the industrial applications. I would like to share four swarm intelligence algorithms mentioned in the article.

2. Ant Colony Optimization

Ant Colony Optimization(ACO) is a graph-based optimization algorithm inspired by how real ants find the shortest path to food. It uses pheromone trails to reinforce good solutions over time, making it highly effective for path planning, routing, and scheduling problems. This makes it a popular choice in network optimization, logistics, and autonomous robot navigation. However, once a path is reinforced, it can be difficult to escape local optima.

3. Particle Swarm Optimization

Particle Swarm Optimization(PSO) is a population-based optimization algorithm inspired by the movement of birds and fish schools. Each particle in the swarm represents a potential solution and moves through the search space by following the best-known solutions. It is widely used in engineering, AI, robotics, and neural network training due to its simplicity and efficiency in solving continuous optimization problems. However, it tends to converge to local optima in highly complex landscapes.

4. Artificial Bee Colony

Artificial Bee Colony(ABC) is an optimization algorithm inspired by the foraging behavior of honeybees. It divides the swarm into employed, onlooker, and scout bees, allowing for a good balance between exploration and exploitation. ABC is effective in function optimization, feature selection, clustering, and scheduling problems. It performs well in high-dimensional search spaces but can sometimes require more computational effort compared to PSO.

5. Cuckoo Search Algorithm

In the Cuckoo Search Algorithm(CSA), three key steps are followed throughout its implementation:

1. Each cuckoo lays an egg (i.e., generates a new candidate solution).
2. The egg is placed in a randomly chosen nest (i.e., replacing an existing solution).
3. Quality eggs (better solutions) are passed down through generations, while host birds may discover and discard some cuckoo eggs with probability p , leading to new explorations.

In robot path planning, each "egg" represents a newly generated potential path, and the nest selection process corresponds to evaluating and replacing existing paths with better alternatives. Over successive iterations, poor paths are discarded while optimal routes are refined, ensuring efficient navigation with obstacle avoidance and multi-robot coordination.

CSA provides strong multi-modal optimization, making it effective for complex problems like multi-robot navigation, coverage optimization and dynamic path adjustments, while requiring fewer parameters to fine-tune.

6. Comments

To explain why CSA is outstanding, we need to know what does **multi-modal** mean. Multi-modal optimization refers to the ability of an algorithm to identify and handle multiple optimal or near-optimal solutions in a given

search space. That means CSA has a stronger ability to find the actual global optimum compared to ACO, PSO and ABC because of its better exploration strategy provided by "*Levy flights*" inside the CSA algorithm.

The lack of widespread adoption of CSA (Cuckoo Search Algorithm) in swarm robotics can be attributed to several factors, including the nature of decentralized control in swarm systems and the challenge of broadcasting a global optimum effectively.

1.Introduction

Task allocation is a critical problem in multi-UAV systems, where multiple autonomous agents must efficiently distribute and execute tasks while considering constraints such as energy, time, and communication. Several methods have been proposed, each with different computational properties and trade-offs. This report compares five major approaches: CBBA-based, CNP-based, Game Theory-based, Optimization-based, and Reinforcement Learning-based methods.

2.Methods Overview

2.1 CBBA Based

The Consensus-Based Bundle Algorithm (CBBA) is a decentralized, greedy algorithm that assigns tasks to UAVs using a scoring mechanism. UAVs construct a bundle of tasks and reach consensus through iterative information exchange. CBBA has been widely applied in search and rescue (SAR) missions, as well as in intelligence, surveillance, and reconnaissance (ISR) and suppression of enemy air defenses (SEAD) military applications. Its advantages include scalability for large UAV groups, high efficiency with low communication overhead, and conflict-free task assignment. However, CBBA can lead to suboptimal solutions and requires multiple consensus iterations to finalize task allocations.

2.2 CNP Based

The Contract Net Protocol (CNP) follows an auction-based approach where a central system, often a UAV, broadcasts tasks, and other UAVs submit proposals. The task announcer then selects the best bid and assigns tasks accordingly. CNP-based methods are highly effective in task reallocation and are well-suited for dynamic environments due to their flexibility and adaptability. However, they heavily depend on the communication scheme among agents, which can lead to a high computational burden. Additionally, message congestion has been identified as a significant challenge in CNP-based implementations.

2.3 Game Theory Based

Game theory-based algorithms model UAVs as rational players optimizing their task selection based on predefined payoff functions, falling into two categories: cooperative and competitive. The computational speed varies depending on the game type, with methods like Nash equilibrium requiring iterative computation. Typically, the algorithm's complexity is $O(N^2)$ or higher, depending on the strategy space. The key advantage of this approach is its suitability for strategic decision-making, allowing for fairness and competition among UAVs. However, it comes with high computational costs, especially in large-scale systems, where complexity increases with swarm density, and convergence to equilibrium is not always guaranteed.

2.4 Metaheuristic Based

Metaheuristics encompass various optimization methods, including genetic algorithms, simulated annealing, and swarm intelligence. **Swarm intelligence**, inspired by natural collective behaviors, is widely used in multi-agent task allocation. While individual agents operate with simple rules, their collective interactions enable the execution of complex tasks efficiently, leading to robust and computationally efficient solutions. However, these algorithms may occasionally assign unnecessary tasks to agents, causing conflicts, and they can exhibit slow global adaptation to environmental changes.

2.5 Reinforcement Learning Based

Reinforcement learning (RL) enables UAVs to learn optimal task allocation policies through continuous interaction with the environment. It offers fast execution speed and adaptability to dynamic and uncertain conditions, allowing UAVs to refine their strategies over time. However, RL requires extensive training data and significant computational resources during training. Additionally, in multi-agent settings, where multiple UAVs operate and learn simultaneously, training can become unstable.

3. Comments

Both CBBA-based and CNP-based approaches fall under auction-based algorithms, relying on explicit communication methods for task allocation. These techniques commonly utilize various network topologies, such as mesh, row, star, circular, and hybrid structures, to facilitate information exchange. In some cases, asynchronous communication schemes are implemented to minimize communication overhead.

In summary, each task allocation method has distinct advantages and trade-offs.

- CBBA-based methods offer fast task allocation with low communication but are greedy and suboptimal.
- CNP-based methods provide flexibility in dynamic environments but require high communication overhead.
- Game Theory-based approaches are useful for competitive and cooperative task allocation but may suffer from computational complexity.
- RL-based approaches are highly adaptive and scalable, but training is computationally intensive.

Paper Summary: Recent trends in robot learning and evolution for swarm robotics

Yeqi Sang

1 Empirical Puzzle

Swarm robotics offers significant advantages in robustness, scalability, and decentralized control. However, designing individual robot behaviors that yield the desired emergent collective behavior remains an open problem. The core research question is: **How can we automatically generate control software that enables a robot swarm to self-organize and achieve mission-specific goals?**

2 Review of Existing Scholarship

Traditionally, robot swarms have been designed manually through trial-and-error, which is time-consuming and expertise-dependent. Several methods are introduced.

2.1 Evolutionary robotics

Neuro-evolution is one of the earliest automatic design methods in swarm robotics. In this approach, neural networks are used as black-box controllers, and the search performed by the evolutionary algorithm does not require domain-specific heuristic information. Two main challenges remain in the fields:

- The challenge of **fitness engineering** in defining appropriate objective functions.
- The **reality gap** between simulation and real-world deployment

2.2 Novelty Search

Novelty search generates a set of behaviorally diverse instances of control software. This approach promises to avoid the issue of deception in objective function engineering. The design method avoids premature convergence by optimizing behavioral diversity instead of the mission-specific objective function. Yet, for more complex collective behaviors, novelty search has not produced control software that performs as well as control software generated with a mission-specific objective function.

2.3 Automatic Modular Design

Automatic modular design methods generate control software that is composed of software modules that are assembled into a more complex control architecture—e.g., finite-state machines or behavior trees. Preliminary results indicate that automatic modular design methods are a viable alternative to neuro-evolutionary design methods, with comparable performance and smaller reality gap. The limitations are:

- A human designer must manually implement the modules with sufficient domain knowledge of the task.
- The choice of modules implicitly restricts the space of possible mission, and later, with more human operations engaged, it will turn into a manual design rather than an automatic one.

2.4 Embodied Evolution

Embodied evolution represents online design process in which each robot in the swarm executes part of a distributed evolutionary algorithm. Embodied evolution provides important advantages with respect to offline approaches. As evolution takes place directly in the mission environment, transferability is no concern. Besides, the distributed nature of the design process allows exploring different solutions in parallel. While, embodied evolution still faces several challenges in the context of the design of control software for robot swarms.

- Robots need more computing resource and hardware support since they need to execute their own control software and the design process.
- Power support and it trends to have higher likelihood of sensor or actuator failures due to the relative long operation time.
- Online evolutionary process can only be achieved if the individuals of the swarm can assess the performance of their chosen genome, that means they need certain global information across the swarm.

2.5 Robot Learning

Multi-agent reinforcement learning represents robots are controlled by an instance of control software in an arbitrary form and a reinforcement learning algorithm is used to optimize the instance of control software according to a mission-specific reward function. The application of multi-agent reinforcement learning poses several challenges that still hinder its application in swarm robotics.

- **spatial credit assignment:** When designing control software using reinforcement learning, the mission designer needs to decompose the reward function of the whole swarm into rewards that can be assigned for individual contributions.
- The joint spaces that are concatenated over the individual action and state spaces of each individual agents, suffer heavily from the curse of dimensionality. Thus, addressing large swarm sizes is infeasible in practice.

Imitation learning is an algorithm that generates control software capable of reproducing a desired behavior, based on provided demonstrations of that behavior. The main two challenges in swarms are:

- Existing methods typically rely on detailed demonstrations, making them suitable only when the desired collective behavior has already been implemented.
- There is no standard way to measure similarity between demonstrated and generated behaviors. Collective-level features are easier to choose but hard to formalize, while individual features are easier to compute but require mission-specific knowledge.

3 Author's Comments

One major challenge in the automatic design of swarm robotics control software is the *reality gap*—the mismatch between simulated and real-world performance. While high-fidelity simulations can reduce this gap, they are computationally expensive and may compromise the desirable interchangeability of robots in a swarm. To address this, future research should focus on design methods that are inherently robust to the reality gap. Approaches such as automatic modular design allow for the integration of domain knowledge into reusable software modules, enhancing transferability. Alternatively, hybrid methods that periodically evaluate control software on real robots or in pseudo-reality contexts may help estimate real-world performance more efficiently. Selecting behaviorally novel or promising solutions for real-world evaluation can further improve cost-effectiveness.

Online design methods offer the potential to eliminate the reality gap by adapting control software during mission execution. However, they require robots to evaluate performance using only local information and must operate safely in unknown environments. A promising strategy involves deploying offline-designed baseline behaviors, which can be tuned or recombined online to meet specific mission requirements. This paradigm enables swarms to autonomously adapt while maintaining safe and functional behavior from the outset, even in dynamic or unpredictable conditions.

As missions grow in complexity, the limitations of traditional objective function-based design become apparent. Missions with multiple or conflicting goals require more advanced techniques such as incremental evolution or curriculum learning to gradually build complex behaviors. Additionally, novelty search and quality diversity algorithms can promote behavioral exploration while avoiding local optima. Beyond objective functions, imitation learning—guided by demonstrations from nature or human teachers—offers a pathway to design robust collective behaviors without explicitly modeling them. These alternative methods may

4 Conclusion

Automatic design methods offer a compelling path forward in swarm robotics, but challenges like the reality gap and objective function engineering must be addressed. This review highlights recent advances and provides guidance for future research in making swarm systems more adaptive, scalable, and transferable to real-world environments.

Summary: Restoring Connectivity in Robotic Swarms – A probabilistic Approach

Author: Yeqi Sang

Puzzle Abstract

The paper addresses the challenge of restoring connectivity in swarm robotic system when individual robots become disconnected due to factors such as hardware failures, unexpected environmental obstacles, or motion errors. Unlike traditional connectivity maintenance techniques that assume a fully controlled environment, the study focuses on a scenario where robots lose connectivity unexpectedly and must be actively searched for and reintegrated into the swarm. The key research point is: How can swarm connectivity be restored efficiently when robots get lost due to motion errors and uncertain localization?

Review of Existing Scholarship

The paper reviews three main approaches to swarm connectivity restoration:

1. **Swarm-State Known:** Methods where the position of all robots is accurately known. These approaches plan optimal paths for lost robots to reconnect to the swarm but fail when positions are uncertain.
2. **Swarm-State Estimated:** Methods that estimate the positions of lost robots using indirect information, such as learned behaviors or environmental models. However, these methods do not account for cumulative motion errors.
3. **Swarm-State Not Used:** Methods such as rendezvous-based (predefined meeting points) and behavior-based (backtracking or random walks) strategies. These methods assume robots can accurately navigate back to connectivity, which is often unrealistic.

Additionally, the study compares swarm connectivity restoration to wireless sensor networks (WSNs) and multi-target search problems, where targets (lost nodes or individuals) are sought within uncertain environments. However, the key distinction is that lost robots can be utilized as active searchers once found, unlike passive targets in traditional search problems.

Author's Argument

The paper introduces a probabilistic connectivity-restoration strategy where a leader robot searches for lost follower robots using probability-based search methods. The approach consists of three key steps:

1. **Identifying optimal Regions-of-Interest:** Using probability theory, the system estimates where lost robots might be and defines high-priority search areas.
2. **Optimizing the inter-region search path:** The leader robot determines the optimal order to visit Regions-of-Interests, using a travelling-salesperson-based algorithm that minimizes total search time.
3. **Executing intra-region searches:** Within each Regions-of-Interest, a probabilistic intra-region search strategy is applied, ensuring that areas with the highest probability of containing lost robots are explored first.

The key novelty of this approach

- Uses probability models to guide search efforts efficiently
- Recruits newly found robots to assist in the search, speeding up the restoration process
- Outperforms traditional random-walk search strategies by minimizing search time.

Research Design

1. **Dependent Variable:** Time and efficiency of connectivity restoration measured by the total distance traveled and the number of robots successfully reconnected.
2. **Independent Variables:** The search strategy used, including the use of probability -based method vs. traditional methods like random walks.
3. **Empirical Strategy:** The proposed approach was validated through simulated experiments. The leader robot follows a systematic probability-based search to locate, and guide lost robots
4. **Comparison Method:** The proposed probabilistic search strategy was compared to a random-walk method to demonstrate its efficiency.

Findings and Results

1. Performance Comparison: The probabilistic approach significantly reduced the total search distance needed to restore connectivity, outperforming the random-walk method in 85% of test cases.
2. Effectiveness of Robot Recruitment: Allowing found robots to assist in searching for further improved efficiency
3. Search Optimization: Using iso-probability curves to model potential lost robot locations minimize unnecessary searches, making the process more effective than brute-force approaches

Summary of Paper - Swarm Robotics for Autonomous Aerial Robots: Features, Algorithms, Control Techniques, and Challenges

Author: Yeqi Sang

1. Introduction

This paper provides a comprehensive review of advancements and emerging trends in Swarm Robotics, with a specific focus on the coordination and control of Swarm Aerial Robots (SARs). It explores key features and characteristics of Swarm Robotics, its integration with Swarm Intelligence, and fundamental control algorithms that enable decentralized, self-organized behaviors in aerial robot swarms. Innovations in swarm navigation and control algorithms facilitate precise coordination, leveraging distributed decision-making among individual robots to achieve shared objectives. Additionally, incorporating principles from differential geometry enhances guidance, navigation, and control, optimizing the performance of SARs. The paper also discusses various swarm control techniques, infrastructure requirements, and challenges associated with implementing Swarm Robotics in Aerial Robots.

2. Infrastructure of Swarm Aerial Robotics (SAR)

The infrastructure of Swarm Aerial Robotics (SARs) consists of several essential components that enable coordination, autonomy, and efficiency in swarm operations. Each autonomous aerial robot is equipped with sensors, processors, and communication modules, allowing real-time data exchange and decision-making. A central or decentralized control unit manages the swarm, ensuring efficient coordination and execution of tasks. A robust communication system, utilizing technologies like Wi-Fi, Bluetooth, or Zigbee, facilitates seamless interaction among robots. Integrated sensor systems such as LiDAR, GPS, IMUs, and cameras enable perception, obstacle avoidance, and situational awareness. Navigation and guidance systems support path planning, collision avoidance, and mission execution, ensuring adaptability in dynamic environments. The power system, primarily based on batteries or tethered power sources, remains a crucial factor in determining the operational endurance of SARs. These elements collectively provide the foundation for SARs to function effectively in applications such as search and rescue, environmental monitoring, and precision agriculture.

3. Challenges of Implementing Swarm Robotics in Flying Robots

Despite the advantages of Swarm Robotics in aerial robots, several challenges hinder their large-scale implementation. Communication reliability remains a major issue, as disrupted or weak signals can affect swarm coordination, especially in complex environments. Control and synchronization of multiple robots in real-time require sophisticated algorithms to manage formation control, cooperative task allocation, and adaptive responses to environmental changes. Power and energy limitations pose another challenge, as aerial robots rely on batteries with limited flight durations, restricting the scalability of long-duration missions. Additionally, scalability and robustness issues emerge when increasing the swarm size, making it difficult to manage decentralized decision-making efficiently. Navigation and perception difficulties, such as Simultaneous Localization and Mapping (SLAM), obstacle avoidance, and path planning, further complicate SAR operations in dynamic environments. Security concerns, including cybersecurity threats and ethical considerations, must also be addressed to ensure the safe and responsible deployment of SARs. Overcoming these challenges requires advancements in energy-efficient control algorithms, adaptive learning mechanisms, and secure communication protocols.

4. Comments

This paper provides a broad overview of Swarm Aerial Robotics (SARs), summarizing its infrastructure and challenges but lacking depth in discussing specific issues. While it introduces key concepts and algorithms, it does not provide detailed examples or technical breakdowns of the challenges faced in real-world applications. The discussion on communication reliability, energy constraints, and scalability remains surface-level, without case studies or concrete scenarios illustrating their impact. As a review, it serves as a good introductory reference, but it lacks the depth needed for researchers seeking technical insights or advanced solutions in swarm aerial robotics.